
EXACT AND FAST SUBSET SELECTION ALGORITHMS FOR THE BI-OBJECTIVE INTEGRAL R_2 INDICATOR

Michael T. M. Emmerich

Faculty of Information Technology, University of Jyväskylä, Finland

ORCID: 0000-0002-7342-2090

Abstract

We study fixed-cardinality subset selection for the exact integral bi-objective R_2 indicator with a uniform continuum of weighted Tchebycheff scalarizing functions. The indicator measures the area under the lower envelope of scalarizing losses over weight space, rather than a finite sample average over weight vectors. For a sorted bi-objective Pareto-front approximation, represented by points ordered by increasing first objective and decreasing second objective, we derive an exact adjacent-neighbor decomposition of this integral objective into boundary terms, unary diagonal corrections, and selected-neighbor transition terms. This yields an exact Bellman dynamic program with $O(kn^2)$ running time for selecting k of n candidate points. We then prove that the transition matrix is Monge. This gives a divide-and-conquer implementation with $O(kn \log n)$ running time and, more strongly, a staircase matrix-search implementation with $O(kn)$ running time under constant-time arithmetic comparisons. The matrix-search proof is presented through a lower-envelope sweep over single-crossing transition functions and includes the triangular feasibility condition $i < j$. The algorithms are exact for the continuous integral R_2 setting and are distinct from finite-weight-vector approximations, although they are related to earlier exact and dynamic-programming work on two-dimensional indicator-based subset selection, including hypervolume subset selection. Reproducible Python code compares exhaustive enumeration, the direct left-to-right dynamic program, the divide-and-conquer dynamic program, and the matrix-search implementation under explicit consistency checks.

Keywords multiobjective optimization; biobjective optimization; integral R_2 indicator; subset selection; Pareto-front approximation; dynamic programming; Monge arrays; matrix search; Tchebycheff scalarization

1 Introduction

Quality indicators are often used to compare or select finite nondominated approximations of a Pareto front. In two-objective minimization, the candidate archive can be sorted by increasing first objective and decreasing second objective. The increasing-decreasing order is introduced only as an algorithmic representation of the nondominated archive. The algorithmic question considered in this paper is the following: given such an archive and a cardinality budget k , which k points should be retained if quality is measured by the *exact integral R_2 indicator*?

The classical R_2 idea evaluates a set through scalarizing utility or loss functions [1]. For a weight $\lambda \in [0, 1]$ and a utopian point z^+ , the bi-objective weighted Tchebycheff loss of a point a is

$$g_\lambda(a; z^+) = \max\{\lambda(a_1 - z_1^+), (1 - \lambda)(a_2 - z_2^+)\}.$$

A set is evaluated via its best point for each weight. The standard finite-weight implementation averages these best losses over a finite set of weights. The *Integral R_2* indicator considered in our work [2, 3, 4] yields a Pareto compliant (rather than weakly Pareto compliant) version of the classical R_2 indicator by integrating

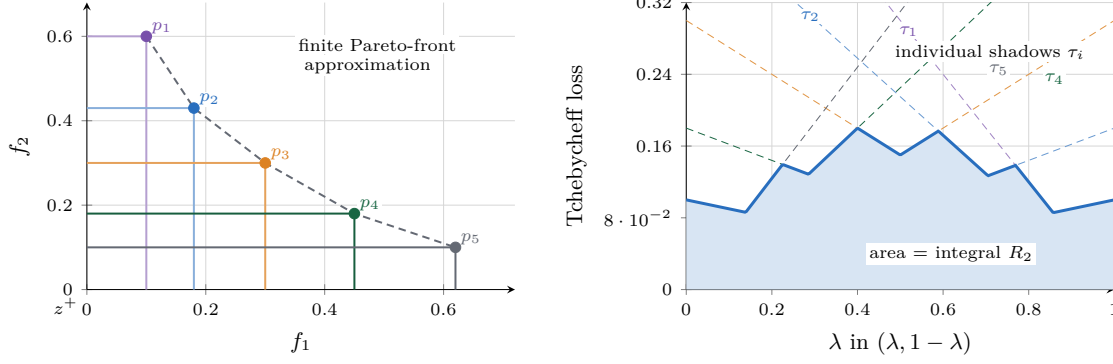


Figure 1: Tchebycheff shadows of a finite Pareto-front approximation. Each archive point defines one shadow function over the weight interval; the exact integral R_2 value is the shaded area under the lower envelope of these shadows.

over the entire weight interval instead of a finite set of weight vectors. After translating z^+ to the origin, the relevant envelope is

$$r_{\mathcal{P}}(\lambda) = \min_{a \in \mathcal{P}} g_{\lambda}(a; 0), \quad R_2(\mathcal{P}) = \int_0^1 r_{\mathcal{P}}(\lambda) d\lambda.$$

Here the integration variable λ moves continuously from full emphasis on the second objective at $\lambda = 0$ to full emphasis on the first objective at $\lambda = 1$. For each value of λ , the archive is first reduced to the point with the smallest weighted Tchebycheff loss; only then is this best loss integrated over all weights. Thus $R_2(\mathcal{P})$ is the area under the lower envelope of the pointwise losses over the whole preference interval, with respect to the uniform measure $d\lambda$. This is different from numerical quadrature or a finite grid of weights: every weight in $[0, 1]$ contributes, and a point contributes only on those subintervals on which its loss curve forms the lower envelope. Since the formulation uses losses, smaller integral area means a better approximation.

Figure 1 shows the finite-archive viewpoint used in this paper. The Tchebycheff-shadow terminology and visualization are adapted from Emmerich [11]. Each point $p_i = (x_i, y_i)$ of a finite Pareto-front approximation casts a Tchebycheff shadow over the weight interval,

$$\tau_i(\lambda) = \max\{\lambda x_i, (1 - \lambda)y_i\}.$$

The set value is obtained by taking the lower envelope of these shadows,

$$r_{\mathcal{P}}(\lambda) = \min_i \tau_i(\lambda),$$

and integrating the shaded area under this envelope. Subset selection can therefore be viewed as retaining only k shadows while preserving the integral envelope as well as possible. This finite-envelope viewpoint is the starting point of the exact bi-objective computation of Schäpermeier and Kerschke [2, 3]. The Tchebycheff-shadow geometric explanation used here follows the terminology and interpretation in Emmerich [11]. Appendix A gives a detailed expository derivation of the elementary integral calculation behind the adjacent-neighbor formula used below. That appendix is included only to make the paper more self-contained; it adds no new result beyond the integral R_2 analysis of Schäpermeier and Kerschke and the shadow interpretation just cited. Closely related integral and exact-calculation questions for the R_2 indicator, including higher-dimensional settings, are also studied independently by Jaszewicz and Zielniewicz [4]. The present paper uses the bi-objective integral setting for fixed-cardinality subset selection.

The subset-selection version of this problem is important for archiving, benchmarking, and postprocessing, and selection in set-oriented and evolutionary search. Archives produced by multiobjective optimizers may contain many nondominated points, while visual inspection, decision-support sessions, or downstream expensive evaluation may require a small representative subset. A finite-weight approximation would turn the problem into a discrete aggregation problem whose outcome depends on the chosen grid of weights. The exact integral criterion removes this discretization parameter in the bi-objective setting and asks for the best cardinality- k subset with respect to the continuous scalarization envelope.

The contributions of this paper are as follows.

- We derive an exact adjacent-neighbor decomposition of the bi-objective integral R_2 value of a sorted selected subset.
- We obtain a Bellman dynamic program for exact fixed-cardinality subset selection in $O(kn^2)$ time.
- We prove a Monge property of the transition matrix and use it to obtain a divide-and-conquer dynamic program with $O(kn \log n)$ running time.
- We sharpen the layer computation to a staircase matrix-search/lower-envelope algorithm with $O(n)$ time per dynamic-programming layer, hence $O(kn)$ total time.
- We provide reproducible computational experiments comparing exhaustive enumeration, direct dynamic programming, divide-and-conquer dynamic programming, and the matrix-search implementation under explicit consistency checks.

The reproducibility material is available from the repository listed in the Code availability section.

The paper is organized as follows. Section 2 places the result in the context of integral R_2 indicators, two-dimensional indicator subset selection, and Monge matrix search. Section 3 introduces the exact integral subset-selection problem and derives the adjacent-neighbor decomposition. Section 4 presents the three exact algorithms in a parallel format: the direct Bellman dynamic program, the divide-and-conquer dynamic program, and the matrix-search dynamic program, each with motivation, pseudocode, correctness, and complexity discussion. Section 5 reports empirical verification tests and CPU-time experiments. Section 6 summarizes the results and gives an outlook. Appendix A gives an expository derivation of the exact bi-objective integral R_2 decomposition used in the main text; it adds no new result, but records the elementary calculation behind the formula. Appendix B gives a gentle introduction to Monge matrix search and the staircase lower-envelope viewpoint.

2 Related work

The R_2 indicator originates in the framework of Hansen and Jaszakiewicz for evaluating approximations to the nondominated set under a distribution of utility functions [1]. In practice, R_2 has often been used through finite weight-vector approximations and in indicator-based search methods, for example, in the R_2 indicator-based multiobjective search framework of Brockhoff, Wagner, and Trautmann [5]. Schäpermeier and Kerschke recently revisited the indicator from the continuous/integral perspective and showed that, for a uniform continuum of Tchebycheff utility functions, the resulting bi-objective indicator can be computed exactly and is Pareto-compliant [2, 3]. Jaszakiewicz and Zielniewicz independently study exact calculation and properties of an integral R_2 multiobjective quality indicator, including higher-dimensional cases [4]. Together, these works motivate treating integral R_2 as an exact continuous indicator rather than as a finite weight-vector average. The present paper uses the bi-objective integral setting as its starting point and studies exact fixed-cardinality subset selection for a sorted Pareto-front approximation.

Dynamic programming and exact algorithms for two-dimensional indicator-based subset selection are not new in themselves. For the hypervolume indicator, Kuhn, Fonseca, Paquete, Ruzika, Duarte, and Figueira studied two-dimensional hypervolume subset selection, including formulations and algorithms for that special geometric setting [6]. Bringmann, Friedrich, and Klitzke also studied two-dimensional subset selection for the hypervolume and epsilon indicators and gave efficient algorithms for those indicators [7]. More recently, Korogi and Tanabe proposed polynomial-time dynamic programs for the bi-objective indicator-based subset selection problem with IGD, IGD⁺, R_2 , and NR2 under practical assumptions [8]. Their conference paper formulates R_2 and NR2 in terms of a finite weight-vector set. The contribution of the present paper should therefore be read narrowly: it provides a self-contained derivation, implementation, and Monge acceleration for exact bi-objective integral R_2 subset selection, i.e., for the specific Pareto compliant integral version of the R_2 indicator formulated in [2, 3] and [4].

The structural motivation also parallels the Monge/submodular-lattice approach used in one-dimensional Riesz-energy subset selection [12]. In the Riesz-energy case, all selected pairs interact, so an ordinary left-to-right Bellman recurrence is not exact; in the integral R_2 case, the objective has only adjacent selected-neighbor terms, and the Bellman recurrence is exact.

Monge arrays and totally monotone matrices are a classical source of faster dynamic programs. The SMAWK algorithm of Aggarwal, Klawe, Moran, Shor, and Wilber computes row or column minima of a totally monotone matrix in linear time under implicit matrix access [9]. The matrix-search result in Section 4.4 uses the same principle but specializes it to the staircase-shaped feasible domain of the recurrence and presents the algorithm as a lower-envelope sweep over single-crossing transition functions.

3 Preliminaries and problem setting

We consider a minimization archive of nondominated points

$$p_i = (x_i, y_i), \quad i = 1, \dots, n,$$

measured relative to a utopian point that has been translated to the origin. The archive is sorted as

$$0 < x_1 < \dots < x_n, \quad y_1 > \dots > y_n > 0.$$

Next, we will introduce the analytic expression of the exact integral R_2 . For a detailed geometrical motivation, we refer to the original papers. For a selected subset $S \subseteq \{1, \dots, n\}$, the exact integral R_2 value for the uniform bi-objective weighted Tchebycheff family is

$$R_2(S) = \int_0^1 \min_{i \in S} \max\{wx_i, (1-w)y_i\} dw. \quad (1)$$

This is the continuous-utility version studied in the bi-objective case by Schäpermeier and Kerschke [2, 3] and, from a broader exact-calculation perspective including higher dimensions, by Jaskiewicz and Zieleniewicz [4]. The present paper focuses on fixed-cardinality subset selection for the bi-objective exact integral objective, rather than on a finite weight-vector approximation.

The subset-selection problem treated here is:

$$\text{minimize } R_2(S) \quad \text{subject to } |S| = k,$$

where k is fixed in advance. Lower values are better. The input order is important: we restrict to one sorted Pareto-front approximation, not to arbitrary unordered two-dimensional point sets.

3.1 Exact adjacent-neighbor decomposition

Let

$$S = (i_1 < i_2 < \dots < i_k)$$

be the selected index vector. Define the corner interaction

$$A_{ij} = \frac{x_j y_i}{2(x_j + y_i)}. \quad (2)$$

The diagonal value A_{ii} is the triangular correction associated with the kink of the single point p_i .

For two consecutive selected points $i < j$, the relevant switch of the lower Tchebycheff envelope is determined by the intersection of the increasing branch $w x_j$ of the right point and the decreasing branch $(1-w)y_i$ of the left point. This occurs at

$$w_{ij} = \frac{y_i}{x_j + y_i}, \quad w_{ij} x_j = (1 - w_{ij}) y_i = \frac{x_j y_i}{x_j + y_i}.$$

Integrating the piecewise-linear lower envelope (cf. Figure 1) and collecting the terms gives the following formula; Appendix A spells out this elementary calculation in detail.

$$R_2(i_1, \dots, i_k) = \frac{x_{i_1}}{2} + \sum_{r=1}^{k-1} A_{i_r, i_{r+1}} + \frac{y_{i_k}}{2} - \sum_{r=1}^k A_{i_r, i_r}. \quad (3)$$

Thus the exact integral R_2 subset objective is an ordered-subset functional: there are left and right boundary terms, unary diagonal corrections, and adjacent selected-neighbor transitions. This adjacent-neighbor decomposition is the key reason why Bellman dynamic programming works here.

4 Exact algorithms

This section presents the three exact dynamic-programming algorithms used in the paper. All three solve the same recurrence derived from the adjacent-neighbor decomposition. They differ only in how the predecessor minimization in one dynamic-programming layer is performed. Throughout the section, let

$$A_{ij} = \frac{x_j y_i}{2(x_j + y_i)}, \quad B_j = A_{jj},$$

and let $P[r, j]$ denote the predecessor stored for backtracking.

4.1 Direct Bellman dynamic programming

The adjacent-neighbor decomposition gives an immediate exact algorithm: build the selected sequence from left to right, and remember only the last selected point. This gives the following Bellman recurrence.

Let $D_r(j)$ denote the best partial value of a selected subsequence of exactly r points that ends in point j , not yet including the final right boundary term $y_j/2$. From (3), the initialization is

$$D_1(j) = \frac{x_j}{2} - A_{jj}, \quad j = 1, \dots, n. \quad (4)$$

For $r = 2, \dots, k$ and $j = r, \dots, n$, the Bellman recurrence is

$$D_r(j) = -A_{jj} + \min_{i < j} \{D_{r-1}(i) + A_{ij}\}. \quad (5)$$

Finally,

$$R_2^*(k) = \min_{j \geq k} \left\{ D_k(j) + \frac{y_j}{2} \right\}. \quad (6)$$

Storing the minimizing predecessor in (5) gives the selected subset by ordinary backtracking.

Algorithm 1 Direct Bellman dynamic program for exact integral R_2 subset selection

Direct-LRDP($p_1, \dots, p_n, k$)

```

    for  $j = 1, \dots, n$  do
         $D_1(j) \leftarrow x_j/2 - B_j$ 
    for  $r = 2, \dots, k$  do
        for  $j = r, \dots, n$  do
             $D_r(j) \leftarrow +\infty$ 
            for  $i = r-1, \dots, j-1$  do
                 $v \leftarrow D_{r-1}(i) + A_{ij} - B_j$ 
                if  $v < D_r(j)$  then  $D_r(j) \leftarrow v$ ,  $P[r, j] \leftarrow i$ 
             $j^* \leftarrow \arg \min_{j \geq k} \{D_k(j) + y_j/2\}$ , with leftmost tie-breaking
        return value  $D_k(j^*) + y_{j^*}/2$  and the backtracked indices
    
```

Walkthrough. The first layer chooses one endpoint j and pays the left boundary term and the diagonal correction. Layer r constructs all best subsequences of cardinality r ending at j by trying every feasible predecessor $i < j$. The final step adds the right boundary term $y_j/2$ and selects the best last endpoint. This is the most transparent implementation of the recurrence and is useful as a correctness reference, but it scans $O(n^2)$ predecessor–endpoint pairs per layer.

Theorem 1 (Correctness of the direct Bellman dynamic program). *Algorithm 1 returns an optimum cardinality- k subset for the exact integral R_2 objective.*

Proof. The proof is by induction over the selected cardinality r . For $r = 1$, the value $D_1(j) = x_j/2 - A_{jj}$ is exactly the left boundary term and diagonal correction for the one-point partial sequence ending at j , without the final right boundary term. Assume that $D_{r-1}(i)$ is the optimum value of every feasible partial sequence of cardinality $r-1$ ending at i . By the adjacent-neighbor decomposition, extending such a sequence by endpoint $j > i$ adds precisely the transition A_{ij} and the new diagonal correction $-A_{jj}$. Taking the minimum over all feasible predecessors therefore gives the best cardinality- r partial sequence ending at j . The final minimization adds the missing right boundary term $y_j/2$, so backtracking the stored predecessors yields an optimum selected sequence. $\square$

Complexity. The direct implementation has k layers and $O(n^2)$ transitions per layer, hence time complexity $O(kn^2)$ and memory $O(kn)$ with backpointers. A memory-reduced value-only version uses $O(n)$ memory.

4.2 Monge transition structure

Further DP speed-ups use Monge structure, which we discuss next. Note that no objective normalization to a common range is required for the Monge argument below. What is needed is positivity after translation relative to the chosen utopian point and the sorted Pareto-front representation. Positive coordinate rescalings,

for example $x \mapsto \alpha x$ and $y \mapsto \beta y$ with $\alpha, \beta > 0$, change the numerical value of the indicator but preserve the Monge property of the transition matrix.

The transition cost (2) is generated by

$$\phi(x, y) = \frac{xy}{2(x+y)}.$$

A direct calculation gives

$$\frac{\partial^2 \phi}{\partial x \partial y} = \frac{xy}{(x+y)^3} > 0.$$

Thus ϕ has increasing differences on the positive quadrant. The sign of this mixed derivative is the analytic property needed for the Monge proof. Consequently, positive multiplicative coordinate rescalings preserve the argument: replacing $\phi(x, y)$ by $\phi(\alpha x, \beta y)$ with $\alpha, \beta > 0$ keeps the mixed derivative positive. Note that scaling may be useful to express preferences, but it is not required for the Monge condition.

Since the front order has $x_j < x_{j'}$ but $y_i > y_{i'}$ whenever $i < i'$ and $j < j'$, the transition matrix satisfies

$$A_{ij} + A_{i'j'} \leq A_{ij'} + A_{i'j}, \quad i < i', j < j'. \quad (7)$$

This is the Monge inequality in the index order used by the archive.

Consequently, the adjacent transition term

$$(i_r, i_{r+1}) \mapsto A_{i_r i_{r+1}}$$

is submodular on the two-coordinate componentwise lattice of increasing index vectors. Summing over adjacent ranks and adding unary and boundary terms preserves submodularity. Therefore, the fixed-cardinality exact integral R_2 subset problem also admits a submodular-lattice interpretation. This mirrors the Monge-to-lattice-to-mincut strategy in the one-dimensional Riesz-energy subset-selection work of Emmerich [12]; there the full Riesz energy contains all selected-pair interactions, so a simple left-to-right Bellman principle is not exact. Here, the exact R_2 decomposition is local along the selected subsequence, so the Bellman recurrence is the most direct polynomial-time algorithm.

The recurrence (5) can be accelerated because the transition costs form a Monge matrix. We first record the standard monotone-predecessor consequence used by the divide-and-conquer implementation. For a fixed layer r , define the implicit transition matrix

$$M_{ij}^{(r)} = D_{r-1}(i) + A_{ij}, \quad i < j. \quad (8)$$

Adding the predecessor value $D_{r-1}(i)$ to row i does not change any Monge difference, so $M^{(r)}$ satisfies the same inequality as A . Therefore, if ties are resolved by choosing the smallest predecessor index, the minimizer

$$\text{opt}_r(j) = \arg \min_{i < j} \{D_{r-1}(i) + A_{ij}\}$$

is nondecreasing in j . Indeed, if $j < j'$ and two candidate predecessors satisfy $i < i'$, the Monge inequality gives

$$M_{ij}^{(r)} + M_{i'j'}^{(r)} \leq M_{ij'}^{(r)} + M_{i'j}^{(r)}.$$

This is the standard exchange inequality behind monotone matrix searching: a later column cannot force the leftmost optimum to move backwards.

4.3 Divide-and-conquer dynamic programming

The monotonicity of $\text{opt}_r(j)$ gives the usual divide-and-conquer computation of one DP layer. To compute $D_r(j)$ for j in an interval $[L, R]$, take the midpoint m , scan only the current admissible predecessor range $[a, b]$ to find $\text{opt}_r(m)$, and recurse on

$$[L, m-1] \times [a, \text{opt}_r(m)] \quad \text{and} \quad [m+1, R] \times [\text{opt}_r(m), b].$$

The triangular condition $i < j$ is handled by replacing the upper scan limit with $\min\{b, m-1\}$. This gives an $O(n \log n)$ worst-case bound for one layer with this simple recursive search. Since there are k selected-cardinality layers, the full divide-and-conquer dynamic program has time bound

$$O(kn \log n).$$

The reference implementation in the reproducibility repository includes both the direct $O(kn^2)$ method and this divide-and-conquer version, and checks that they return the same selected subset and value.

Algorithm 2 Divide-and-conquer dynamic program using monotone predecessors

DCDP-Layer(r, L, R, a, b)
 if $L > R$ return
 $m \leftarrow \lfloor (L + R)/2 \rfloor$
 $u \leftarrow \min\{b, m - 1\}$
 find the leftmost $i^* \in \{a, \dots, u\}$ minimizing $D_{r-1}(i) + A_{im} - B_m$
 $D_r(m) \leftarrow D_{r-1}(i^*) + A_{i^*m} - B_m$, $P[r, m] \leftarrow i^*$
DCDP-Layer($r, L, m - 1, a, i^*$)
DCDP-Layer($r, m + 1, R, i^*, b$)
Divide-Conquer-DP($p_1, \dots, p_n, k$)
 initialize $D_1(j) \leftarrow x_j/2 - B_j$ for $j = 1, \dots, n$
 for $r = 2, \dots, k$ do
 call **DCDP-Layer**($r, r, n, r - 1, n - 1$)
 finish by minimizing $D_k(j) + y_j/2$ and backtracking

Walkthrough. The Monge property implies that the leftmost optimal predecessor is nondecreasing as the endpoint j moves from left to right. The divide-and-conquer algorithm evaluates the middle endpoint of an interval first. Once the optimal predecessor for the middle endpoint is known, all endpoints to the left need only search predecessors no larger than that value, and all endpoints to the right need only search predecessors no smaller than that value. This recursively shrinks the search ranges. The recurrence values are exactly the same as in the direct Bellman dynamic program; only the order and range of predecessor scans change.

Theorem 2 (Correctness and complexity of divide-and-conquer DP). *The divide-and-conquer dynamic program computes the same values as the direct Bellman recurrence. Its running time is $O(kn \log n)$ and its memory usage is $O(kn)$ with backpointers, or $O(n)$ for values only.*

Proof. The Monge inequality and leftmost tie-breaking imply that the optimal predecessor index is nondecreasing in the endpoint j . Therefore, after the midpoint of a column interval has been solved, the predecessor search range for the left recursive subproblem can be restricted to predecessors no larger than the midpoint optimum, and the search range for the right recursive subproblem can be restricted to predecessors no smaller than it. These restrictions do not remove any optimal predecessor, so the values are identical to those of the direct recurrence. In one layer, each recursion level scans $O(n)$ candidate predecessor–endpoint pairs in total, and there are $O(\log n)$ levels. Thus one layer costs $O(n \log n)$, and k layers cost $O(kn \log n)$. $\square$

4.4 Matrix-search dynamic programming

The divide-and-conquer method uses only the monotonicity of the minimizing predecessor. The same Monge structure gives a stronger result: each dynamic-programming layer can be computed in linear time. This can be viewed as a staircase version of monotone matrix search. Classical matrix-search algorithms, including SMAWK, compute minima of totally monotone matrices in linear time under implicit matrix access [9]. Here the feasible matrix is not rectangular, because column j only permits predecessors $i < j$. We therefore give a direct lower-envelope version of the matrix search, specialized to the analytic transition functions of integral R_2 .

4.4.1 Preconditions

For a fixed layer r , recall the implicit transition matrix

$$M_{ij}^{(r)} = D_{r-1}(i) + A_{ij}.$$

The feasible domain is the staircase

$$\Omega_r = \{(i, j) : r - 1 \leq i < j \leq n\}.$$

Thus the feasible rows of column j form the prefix $r - 1, \dots, j - 1$. The matrix-search argument uses four preconditions: entries can be compared in constant time; feasible rows are nested prefixes; every feasible 2×2

submatrix satisfies the Monge inequality; and ties are resolved by the smallest predecessor index. Padding infeasible entries by $+\infty$ is not the right formal model, because it may introduce artificial ties. The correct object is a staircase partial matrix.

4.4.2 Algorithm description

For a fixed predecessor i , define

$$f_i(x) = D_{r-1}(i) + \frac{xy_i}{2(x+y_i)}. \quad (9)$$

Then the transition part of the Bellman recurrence is

$$\min_{i < j} \{D_{r-1}(i) + A_{ij}\} = \min_{i < j} f_i(x_j). \quad (10)$$

The layer algorithm sweeps $j = r, r+1, \dots, n$ from left to right. Before answering the query at x_j , it inserts the newly feasible predecessor $i = j-1$. The active predecessors are stored as the lower envelope of the functions f_i . The envelope is represented by a stack of rows together with crossing thresholds. A pointer moves forward along this stack as the query abscissae x_j increase.

For two rows $i < i'$, set $\Delta = D_{r-1}(i') - D_{r-1}(i)$. Since $y_i > y_{i'}$, the crossing at which the later row i' becomes as good as row i is determined by

$$\frac{x^2(y_i - y_{i'})}{2(x+y_i)(x+y_{i'})} = \Delta. \quad (11)$$

If $\Delta \leq 0$, the later row is already no worse at the origin and the crossing threshold is 0. If $\Delta \geq (y_i - y_{i'})/2$, the later row never becomes strictly better for finite x . Otherwise, (11) has a unique positive solution, obtained from a quadratic equation. At the crossing itself the older row is kept, matching leftmost tie-breaking.

When a new row is inserted, its crossing with the last row on the stack is computed. If the new row never becomes better, it is discarded. If it becomes better before or at the point where the last row would start its own interval of strict optimality, the last row has no remaining open interval on the envelope and is popped. The test is repeated until the new row can be pushed or discarded. At a query x_j , the pointer is advanced while the next envelope row is strictly better at x_j . The current row is then the minimizing predecessor for column j .

Algorithm 3 Matrix-search dynamic program by lower-envelope sweep

Cross(i, h)

return the first abscissa x at which row h is strictly better than row i
 for $f_i(x) = D_{r-1}(i) + xy_i/(2(x+y_i))$; return $+\infty$ if this never occurs

Envelope-Layer(r)

initialize an empty stack of pairs (i, α) , where row i is active from $x = \alpha$
 initialize the query pointer to the first stack element

for $j = r, \dots, n$ **do**

$h \leftarrow j-1$ new predecessor row entering the staircase

$\alpha \leftarrow -\infty$

while stack is nonempty **do**

 let (i, β) be the last stack element

$\alpha \leftarrow \text{Cross}(i, h)$

if $\alpha \leq \beta$ **then** pop the last stack element **else break**

if stack is empty **then** push $(h, -\infty)$

else if $\alpha < +\infty$ **then** push (h, α)

 advance the query pointer while the next stack interval starts at or before x_j

 let i^* be the row at the query pointer

$D_r(j) \leftarrow D_{r-1}(i^*) + A_{i^*j} - B_j$, $P[r, j] \leftarrow i^*$

Matrix-Search-DP($p_1, \dots, p_n, k$)

initialize $D_1(j) \leftarrow x_j/2 - B_j$ for $j = 1, \dots, n$

for $r = 2, \dots, k$ **do** call **Envelope-Layer**(r)

finish by minimizing $D_k(j) + y_j/2$ and backtracking

Walkthrough. In one layer, every feasible predecessor i defines a curve

$$f_i(x) = D_{r-1}(i) + \frac{xy_i}{2(x+y_i)}.$$

Endpoint j queries the lower envelope of the active curves at x_j . As j increases, one new predecessor row $h = j - 1$ enters the feasible set. Because any two curves cross at most once, the lower envelope can be stored as an ordered stack of rows together with the abscissa from which each row becomes active. When a new row enters, it may destroy a suffix of the old envelope; those rows are popped. It is then inserted at its first strict crossing point with the surviving last row. Since both row insertion and query positions move from left to right, each row is pushed once and popped at most once, and the query pointer never moves backwards. Thus one layer takes linear time while computing the same Bellman values as the direct recurrence.

4.4.3 Correctness

The proof combines total monotonicity with the single-crossing structure of the functions (9).

Lemma 1 (Staircase total monotonicity). *For a fixed layer r , consider rows $i < i'$ and columns $j < j'$ such that all four entries belong to Ω_r . If*

$$M_{i'j}^{(r)} \leq M_{ij}^{(r)},$$

then

$$M_{i'j'}^{(r)} \leq M_{ij'}^{(r)}.$$

Proof. The row offsets $D_{r-1}(i)$ preserve the Monge inequality. Hence

$$M_{ij}^{(r)} + M_{i'j'}^{(r)} \leq M_{ij'}^{(r)} + M_{i'j}^{(r)}.$$

Rearranging gives

$$M_{i'j'}^{(r)} - M_{ij'}^{(r)} \leq M_{i'j}^{(r)} - M_{ij}^{(r)}.$$

The right-hand side is nonpositive by assumption. Thus the later row remains no worse in the later column. Only feasible entries of the staircase matrix are used. $\square$

Lemma 2 (Single crossing). *For $i < i'$, the difference $f_i(x) - f_{i'}(x)$ is strictly increasing for $x > 0$.*

Proof. The derivative of the nonlinear part of f_i is

$$\frac{d}{dx} \left(\frac{xy}{2(x+y)} \right) = \frac{y^2}{2(x+y)^2}.$$

Since $i < i'$ implies $y_i > y_{i'}$, and $y/(x+y)$ is strictly increasing in y for every fixed $x > 0$, we have

$$\frac{y_i^2}{2(x+y_i)^2} > \frac{y_{i'}^2}{2(x+y_{i'})^2}.$$

Therefore $(f_i - f_{i'})'(x) > 0$. Hence two rows cross at most once, and after a later row becomes better it remains better for all larger x . $\square$

Theorem 3 (Linear-time layer computation). *For any fixed layer r , all values $D_r(j)$, $j = r, \dots, n$, can be computed from D_{r-1} in $O(n)$ time, assuming constant-time arithmetic comparisons and crossing computations.*

Proof. By the single-crossing lemma, each new row has at most one crossing with each row currently on the envelope. The stack insertion rule is the standard lower-envelope update for single-crossing functions: a row is popped exactly when the new row becomes better no later than the popped row would have become strictly optimal. After the insertion terminates, the stack stores the active envelope rows in increasing predecessor order and the crossing thresholds are increasing. Because the query points x_j are increasing, the query pointer never moves left. Each row is inserted once and popped at most once, and the pointer advances at most once per surviving envelope interval. Thus the total work in one layer is $O(n)$. The row returned at query x_j minimizes (10); subtracting A_{jj} gives the Bellman value (5). Since switching occurs only when a later row is strictly better, ties are resolved by the smallest predecessor index. $\square$

4.4.4 Complexity

Applying the theorem to all $k - 1$ nontrivial layers gives an exact $O(kn)$ time algorithm for fixed-cardinality integral R_2 subset selection in the bi-objective setting. With backpointers the memory usage is $O(kn)$; if only the optimum value is needed, two value layers suffice and memory is $O(n)$. The reference implementation in the reproducibility repository includes this method. It uses floating-point arithmetic with small tolerances and checks it against the direct and divide-and-conquer dynamic programs on the reproducibility cases.

Relation to two-dimensional hypervolume subset selection. A similar matrix-search viewpoint also applies to the classical two-dimensional hypervolume subset-selection problem. For a sorted biobjective Pareto-front approximation, the hypervolume contribution of a selected sequence admits a local dynamic-programming recurrence with transition term of the bilinear form $-x_i y_j$, up to row and column offsets. This transition matrix is Monge or anti-Monge, depending on whether the problem is written as a minimization or maximization recurrence, and therefore the same monotone-predecessor/matrix-search principle yields $O(kn)$ time after sorting. This does not improve the best known two-dimensional hypervolume subset-selection bound $O((n-k)k + n \log n)$ [7, 6], but it shows that the present integral R_2 result fits a broader pattern: in two objectives, exact indicator subset selection often becomes fast when the indicator decomposes into local predecessor-successor terms with a Monge transition structure.

4.5 Complexity summary for the three algorithms

The improvement should be interpreted per dynamic-programming layer. The direct recurrence spends $O(n^2)$ time in each layer and therefore $O(kn^2)$ time overall. Monge monotonicity reduces one layer to $O(n \log n)$ by divide-and-conquer search, giving $O(kn \log n)$ total time. Thus, for fixed or small k , the accelerated method is nearly linear in n up to a logarithmic factor. For balanced subset sizes, however, $k = \Theta(n)$, so the same bound becomes

$$O(n^2 \log n).$$

It is therefore not correct to describe the full algorithm simply as $O(n \log n)$ unless k is treated as a fixed constant.

The asymptotic regimes are summarized in Table 1.

Regime	Direct DP	Divide-and-conquer DP	Matrix-search DP
fixed k	$O(n^2)$	$O(n \log n)$	$O(n)$
$k = \Theta(n)$	$O(n^3)$	$O(n^2 \log n)$	$O(n^2)$

Table 1: Complexity interpretation in two common asymptotic regimes. The divide-and-conquer bound is $O(n \log n)$ per layer and $O(kn \log n)$ overall; the matrix-search bound is $O(n)$ per layer and $O(kn)$ overall.

5 Computational study

The experiments have two purposes. First, they verify that the three dynamic-programming implementations agree with exhaustive enumeration on instances where exhaustive enumeration is feasible. Second, they illustrate the empirical scaling of the direct, divide-and-conquer, and matrix-search implementations in balanced and fixed-cardinality regimes.

5.1 Illustrative seven-point instance

Figure 2 shows the seven-point staircase instance used in the Python test file. For $k = 3$, the dynamic program selects points 1, 5, 7.

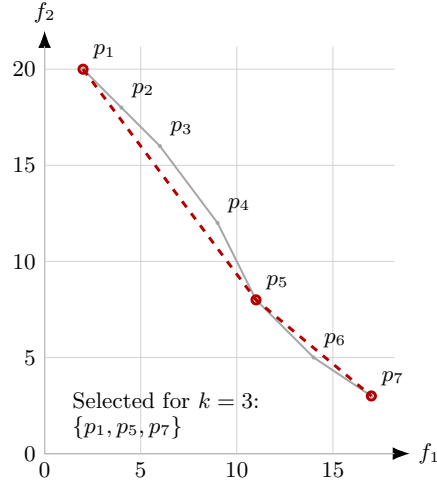


Figure 2: A sorted bi-objective Pareto front. The red-circled points are the optimum cardinality-three subset under the exact integral R_2 objective for this instance.

5.2 Illustrative twenty-point instance

Figure 3 shows a twenty-point Pareto-front approximation. For $k = 10$, the dynamic program selects points

$$\{p_1, p_4, p_6, p_7, p_8, p_9, p_{10}, p_{12}, p_{14}, p_{17}\},$$

with exact value $R_2 = 4.936582464$. This instance is also included in the reproducibility material and is verified by exhaustive enumeration.

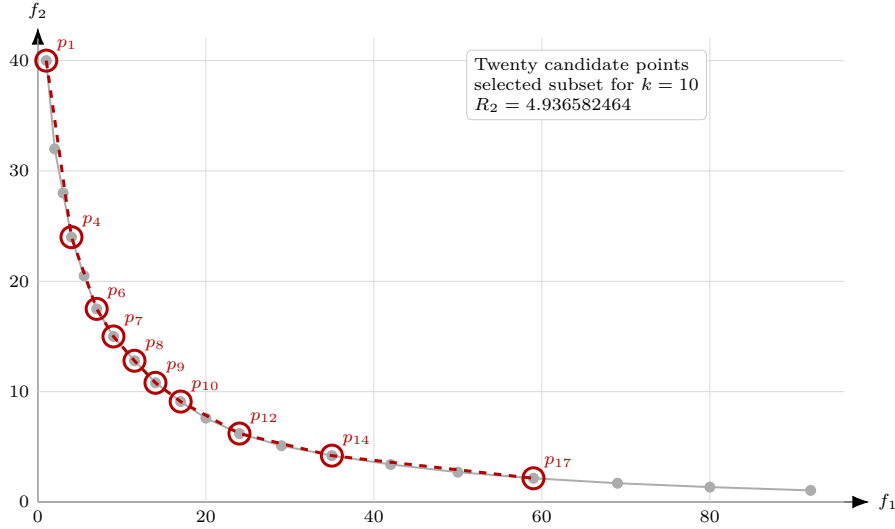


Figure 3: A larger Pareto-front approximation with twenty candidate points. The red-circled points are the optimum cardinality-ten subset under the exact integral R_2 objective for this instance.

5.3 Empirical verification tests

The reproducibility material in the repository implements the direct recurrence (5), the divide-and-conquer accelerated recurrence, the matrix-search recurrence from Section 4.4, exact value evaluation by (3), and

brute-force verification for the displayed examples. Representative output is shown in Table 2. The same material also includes the runtime experiment summarized in Section 5.4. The final two blocks add a twelve-point instance with larger cardinalities and a twenty-point graphical instance; in both cases, the dynamic-programming result is checked against complete enumeration. The repository also includes an eighty-point non-brute-force check comparing the direct, divide-and-conquer, and matrix-search implementations.

Instance	k	selected one-based indices	exact R_2
Seven-point staircase	2	(1, 6)	4.866450887
Seven-point staircase	3	(1, 5, 7)	4.268506714
Seven-point staircase	4	(1, 3, 5, 7)	4.105253002
Seven-point staircase	5	(1, 3, 5, 6, 7)	4.020420466
Six-point mildly irregular	3	(1, 4, 6)	2.824432278
Five-point convex	3	(2, 4, 5)	2.783431481
Twelve-point larger- k front	6	(2, 4, 5, 6, 8, 11)	3.674859356
Twelve-point larger- k front	7	(2, 4, 5, 6, 7, 9, 12)	3.603086767
Twelve-point larger- k front	8	(1, 3, 4, 5, 6, 7, 9, 12)	3.546608278
Twelve-point larger- k front	9	(1, 3, 4, 5, 6, 7, 8, 10, 12)	3.501284795
Twenty-point graphical front	8	(1, 4, 6, 7, 9, 11, 13, 16)	5.035586573
Twenty-point graphical front	10	(1, 4, 6, 7, 8, 9, 10, 12, 14, 17)	4.936582464
Twenty-point graphical front	12	(1, 3, 5, 6, 7, 8, 9, 10, 11, 12, 14, 17)	4.863355081

Table 2: All displayed dynamic-programming solutions are verified by exhaustive enumeration in the reproducibility material; the direct, divide-and-conquer, and matrix-search implementations also agree.

5.4 Runtime assessment with CPU times

To complement the asymptotic discussion, the reference implementation in the reproducibility repository measures CPU wall-clock times using `time.perf_counter()` on deterministic fronts. The benchmark compares exhaustive enumeration, the direct left-to-right dynamic program (LRDP), the divide-and-conquer dynamic program (DC DP), and the matrix-search dynamic program. Exhaustive enumeration is run with a one-second wall-clock time limit per case. All dynamic-programming variants are checked against each other for every benchmark case. When exhaustive enumeration finishes within the time limit, its selected subset and value are also checked against the DP result; otherwise the table records the exhaustive entry as > 1000 ms. The DP timings are averages over ten runs. The timings are intended as reproducibility-oriented reference figures from the unoptimized pure-Python code, not as tuned implementation benchmarks.

Table 3 reports balanced instances with $k = n/2$. The exhaustive runtime grows quickly with the number of subsets, while all dynamic-programming variants remain below a millisecond throughout this range. Figure 4 visualizes the same data on a logarithmic vertical axis. Time-limited exhaustive entries are shown at the one-second cap.

n	k	$\binom{n}{k}$	Exhaustive (ms)	LRDP (ms)	DC DP (ms)	Matrix (ms)
8	4	70	0.233	0.056	0.060	0.082
10	5	252	0.983	0.061	0.058	0.099
12	6	924	3.267	0.077	0.075	0.135
14	7	3,432	13.505	0.107	0.104	0.181
16	8	12,870	55.671	0.148	0.128	0.236
18	9	48,620	229.944	0.205	0.170	0.309
20	10	184,756	> 1000	0.364	0.274	0.457

Table 3: Mean wall-clock runtimes in milliseconds for balanced cases $k = n/2$. Exhaustive enumeration has a one-second time limit per case.

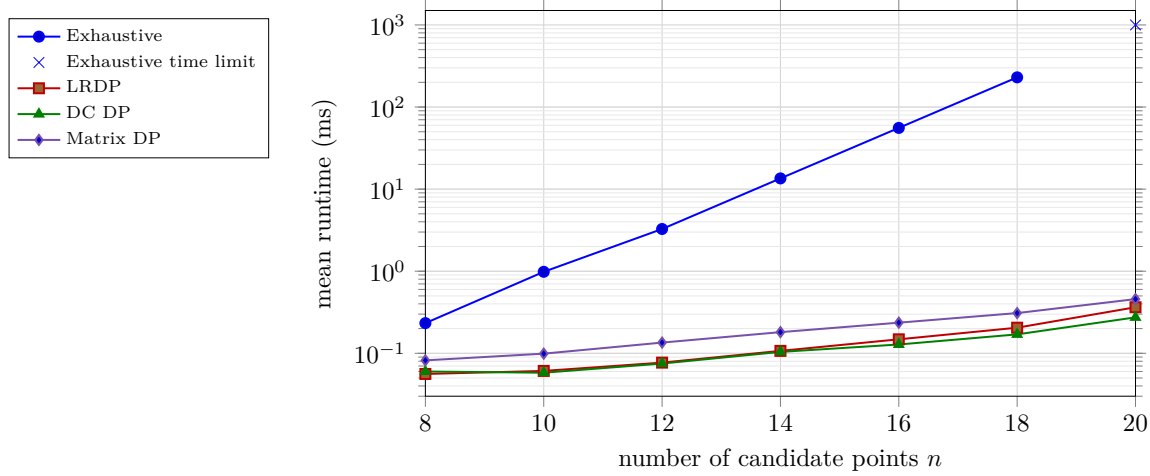


Figure 4: CPU runtime comparison on deterministic balanced fronts with $k = n/2$. The cross marks a case where exhaustive enumeration reached the one-second time limit.

Table 4 repeats the experiment for a small constant cardinality, here $k = 6$, and extends the range to $n = 100$. This illustrates the fixed- k regime discussed in Table 1: exhaustive enumeration grows like $\binom{n}{6} = \Theta(n^6)$, while the dynamic programs grow gently. Figure 5 gives the corresponding plot, with capped exhaustive entries shown at the one-second line.

n	k	$\binom{n}{k}$	Exhaustive (ms)	LRDP (ms)	DC DP (ms)	Matrix (ms)
10	6	210	0.775	0.058	0.066	0.116
14	6	3,003	12.066	0.104	0.097	0.167
18	6	18,564	67.555	0.156	0.229	0.411
22	6	74,613	288.046	0.233	0.242	0.299
26	6	230,230	870.697	0.306	0.190	0.339
30	6	593,775	> 1000	0.411	0.235	0.390
40	6	3,838,380	> 1000	0.751	0.438	0.553
60	6	50,063,860	> 1000	1.625	0.513	1.005
80	6	300,500,200	> 1000	2.858	1.128	1.053
100	6	1,192,052,400	> 1000	4.408	0.922	1.327

Table 4: Mean wall-clock runtimes in milliseconds for the fixed-cardinality regime $k = 6$, extended to $n = 100$. Exhaustive enumeration has a one-second time limit per case.

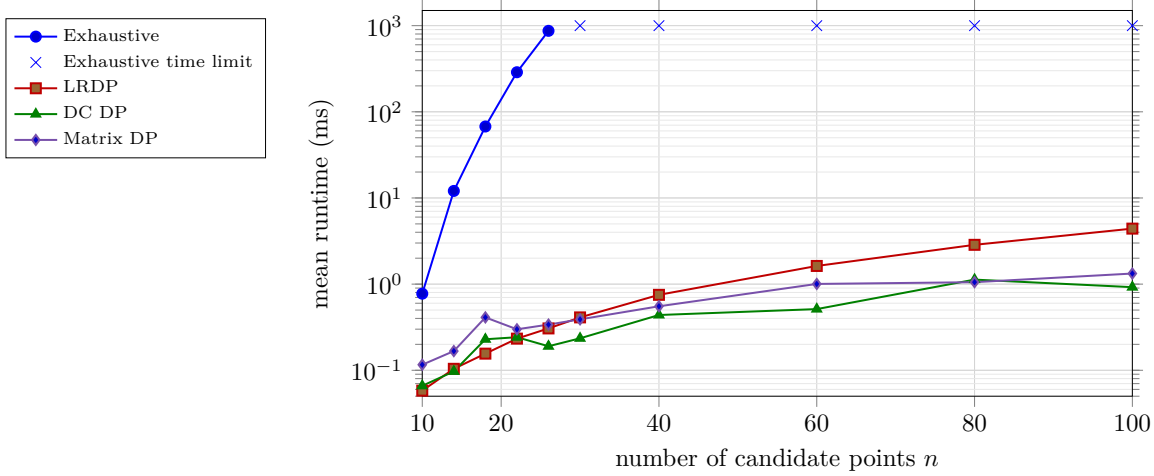


Figure 5: CPU runtime comparison for a small constant cardinality, $k = 6$, up to $n = 100$. All three dynamic-programming implementations are checked against each other for every displayed case.

6 Summary and outlook

This paper has derived exact dynamic-programming algorithms for fixed-cardinality subset selection under the bi-objective integral R_2 indicator. The central observation is that, for a sorted Pareto-front approximation, the continuous Tchebycheff-envelope integral decomposes into boundary terms, unary diagonal corrections, and adjacent selected-neighbor transitions. This gives a direct Bellman recurrence with $O(kn^2)$ running time. The transition matrix is Monge, so predecessor indices are monotone and a divide-and-conquer implementation reduces the cost to $O(kn \log n)$. The stronger matrix-search result uses the staircase totally monotone structure of each layer and a lower-envelope sweep to compute each layer in $O(n)$ time, giving an exact $O(kn)$ algorithm under constant-time arithmetic comparisons. The reference implementation in the reproducibility repository verifies the direct, divide-and-conquer, and matrix-search dynamic programs against exhaustive enumeration where feasible and reports benchmark data under explicit time limits.

The result should be interpreted in the exact integral setting. It does not rely on a finite weight-vector discretization, and no normalization to $[0, 1]$ is required for the Monge condition. What is required is positivity relative to the chosen utopian point and a sorted representation of a two-dimensional Pareto-front approximation. Positive coordinate rescalings preserve the Monge proof, although they change the numerical indicator values and therefore remain a modeling choice.

Several directions remain open. First, other weight densities or scalarizing families may admit similar ordered-subset decompositions, but the transition formula and Monge proof would have to be derived separately. Second, the matrix-search proof suggests looking for other indicator subset-selection problems whose dynamic-programming layers are staircase Monge or totally monotone matrices. Third, the higher-dimensional exact integral R_2 setting studied by Jaszkievicz and Zielniewicz suggests an important extension target. The present dynamic program is inherently bi-objective: the weight domain is the interval $[0, 1]$, the lower envelope is one-dimensional and piecewise linear, and sorted Pareto-front approximations have a predecessor-successor structure. For three or more objectives, the weight domain is a simplex and the lower envelope becomes a higher-dimensional polyhedral complex. A selected point can interact through facets with many other points, so the adjacent-neighbor decomposition used here need not survive. Understanding whether special higher-dimensional Pareto-front geometries, fixed dimension, or computational-geometry representations of the Tchebycheff envelope lead to exact subset-selection algorithms is a natural open problem. Finally, the algorithm suggests a practical archive-reduction tool for exact integral R_2 benchmarking and decision-support pipelines; integrating it into multiobjective optimization software and comparing it with hypervolume-based subset selection are natural next experimental steps.

Code availability

The reproducibility material is available at

<https://github.com/emmerichmtm/integral-r2-subset-selection>. The repository contains the Python reference implementation, the consistency tests comparing exhaustive enumeration and the dynamic programs, and the scripts used to generate the runtime tables.

Acknowledgements

This research is related to the thematic research area Decision Analytics utilizing Causal Models and Multiobjective Optimization (DEMO, jyu.fi/demo) of the University of Jyväskylä.

Declaration on the use of generative AI

Generative AI tools were used for editorial assistance, LaTeX and Python drafting support, and consistency checking during manuscript preparation. All mathematical statements, proofs, algorithms, experiments, citations, and final wording were reviewed by the author, who remains fully responsible for the content of the manuscript.

A Derivation of the exact bi-objective integral R_2 decomposition

This appendix gives a detailed derivation of the closed-form expression for the exact integral bi-objective R_2 value used in the main text. It is included for exposition and for notational self-containment only. It does not add a new mathematical result. The integral viewpoint is due to the exact bi-objective R_2 analysis of Schäpermeier and Kerschke [2, 3]; the Tchebycheff-shadow terminology and geometric interpretation used in Figure 1 follows Emmerich [11].

We use the notation of Figure 1. Each archive point

$$p_i = (x_i, y_i)$$

defines a Tchebycheff shadow over the weight interval,

$$\tau_i(w) = \max\{wx_i, (1-w)y_i\}, \quad 0 \leq w \leq 1.$$

For a selected subset S , the exact integral R_2 value is the area under the lower envelope of these shadows:

$$R_2(S) = \int_0^1 \min_{i \in S} \tau_i(w) dw = \int_0^1 \min_{i \in S} \max\{wx_i, (1-w)y_i\} dw.$$

Thus, for each weight w , one first chooses the selected point with smallest Tchebycheff loss, and only after this pointwise minimization is the result integrated over w .

Figure 6 further illustrates the terminology used below. In objective space, the weighted Tchebycheff sublevel sets are rectangles anchored at the translated utopian point z^+ . In weight space, the same point gives the shadow function $\tau_i(w)$; this is the object whose lower envelope is integrated.

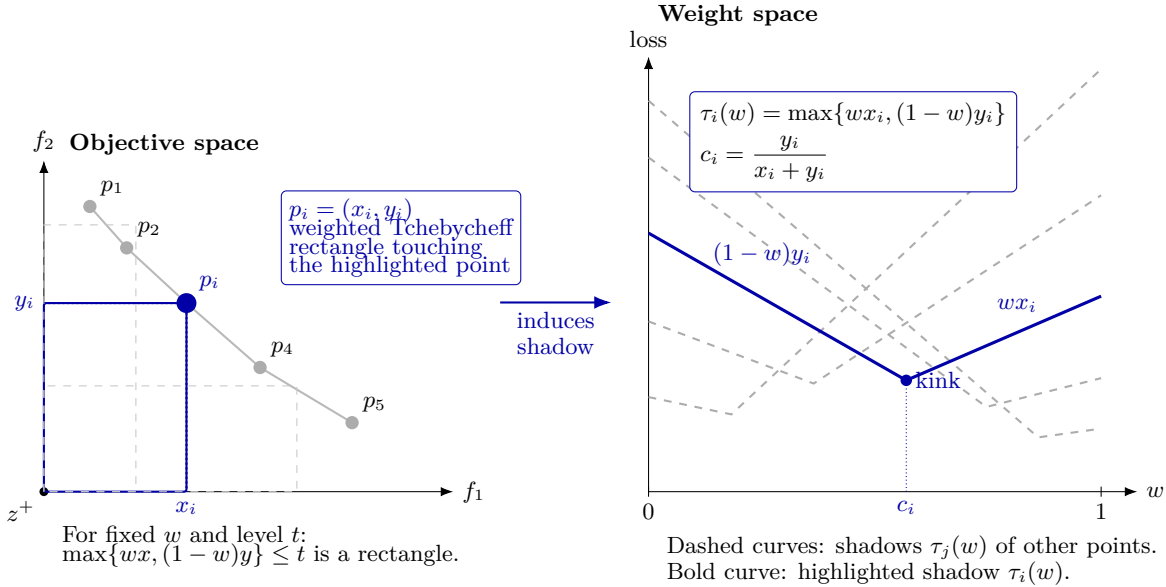


Figure 6: Objective-space and weight-space views of a Tchebycheff shadow. The left panel shows a sorted nondominated chain and weighted Tchebycheff sublevel rectangles anchored at the translated utopian point z^+ . The right panel shows the corresponding shadow curves $\tau_j(w)$ over the weight interval. The highlighted point $p_i = (x_i, y_i)$ gives the bold shadow $\tau_i(w) = \max\{wx_i, (1-w)y_i\}$, with kink at $c_i = y_i/(x_i + y_i)$.

We assume throughout that the archive is a sorted nondominated chain,

$$0 < x_1 < \dots < x_n, \quad y_1 > \dots > y_n > 0.$$

Let the selected subset be written as

$$S = (i_1 < i_2 < \dots < i_k).$$

We shall derive

$$R_2(i_1, \dots, i_k) = \frac{x_{i_1}}{2} + \sum_{r=1}^{k-1} A_{i_r i_{r+1}} + \frac{y_{i_k}}{2} - \sum_{r=1}^k A_{i_r i_r},$$

where

$$A_{ij} = \frac{x_j y_i}{2(x_j + y_i)}.$$

The shadow of a single point

For a single point $p_i = (x_i, y_i)$, the shadow is

$$\tau_i(w) = \max\{wx_i, (1-w)y_i\}.$$

It consists of two line segments. The branch

$$(1-w)y_i$$

is decreasing in w , while the branch

$$wx_i$$

is increasing in w . Their intersection is determined by

$$wx_i = (1-w)y_i.$$

Solving gives

$$wx_i = y_i - wy_i,$$

hence

$$w(x_i + y_i) = y_i,$$

and therefore

$$c_i = \frac{y_i}{x_i + y_i}.$$

At this weight,

$$c_i x_i = (1 - c_i) y_i = \frac{x_i y_i}{x_i + y_i}.$$

The value c_i is the kink position of the shadow τ_i .

The integral of this single shadow is

$$\int_0^1 \tau_i(w) dw = \int_0^{c_i} (1-w)y_i dw + \int_{c_i}^1 wx_i dw.$$

The two parts are

$$\int_0^{c_i} (1-w)y_i dw = y_i \left(c_i - \frac{c_i^2}{2} \right),$$

and

$$\int_{c_i}^1 wx_i dw = \frac{x_i}{2} (1 - c_i^2).$$

Substituting $c_i = y_i/(x_i + y_i)$ and simplifying gives

$$\int_0^1 \tau_i(w) dw = \frac{x_i}{2} + \frac{y_i}{2} - \frac{x_i y_i}{2(x_i + y_i)}.$$

It is therefore natural to define

$$A_{ii} = \frac{x_i y_i}{2(x_i + y_i)}.$$

Then the one-point case reads

$$R_2(\{i\}) = \frac{x_i}{2} + \frac{y_i}{2} - A_{ii}.$$

This calculation explains the diagonal correction term. It is the amount by which the area under the shadow differs from the sum of the two elementary boundary triangles.

The switch between two consecutive selected points

Now consider two selected points $i < j$. Since the archive is sorted nondominated,

$$x_i < x_j, \quad y_i > y_j.$$

Thus point i is better in the first objective, while point j is better in the second objective. On the weight interval this means that point j is favoured near $w = 0$, and point i is favoured near $w = 1$.

The switch of the lower envelope between these two consecutive selected points is determined by the intersection of the decreasing branch of point i and the increasing branch of point j :

$$(1 - w)y_i = wx_j.$$

This is the crossing relevant for the lower envelope between consecutive selected points. Solving gives

$$w_{ij} = \frac{y_i}{x_j + y_i}.$$

At this switch,

$$w_{ij}x_j = (1 - w_{ij})y_i = \frac{x_j y_i}{x_j + y_i}.$$

We define the corresponding corner term by

$$A_{ij} = \frac{x_j y_i}{2(x_j + y_i)}.$$

This is one half of the switch height. These terms are the adjacent selected-neighbor transition terms in the final expression.

The active interval of a selected point

Let

$$S = (i_1 < i_2 < \dots < i_k)$$

be the selected chain. Define the switch weights between consecutive selected points by

$$\alpha_r = w_{i_r i_{r+1}} = \frac{y_{i_r}}{x_{i_{r+1}} + y_{i_r}}, \quad r = 1, \dots, k-1.$$

For notational convenience, set

$$\alpha_0 = 1, \quad \alpha_k = 0.$$

Then the selected point i_r is active on the lower envelope over the interval

$$[\alpha_r, \alpha_{r-1}].$$

This convention is a consequence of the orientation of the weight interval: small w emphasizes the second objective, while large w emphasizes the first objective.

The kink position of the active shadow is

$$c_{i_r} = \frac{y_{i_r}}{x_{i_r} + y_{i_r}}.$$

This kink lies inside the active interval. Indeed, since $x_{i_r} < x_{i_{r+1}}$,

$$\alpha_r = \frac{y_{i_r}}{x_{i_{r+1}} + y_{i_r}} < \frac{y_{i_r}}{x_{i_r} + y_{i_r}} = c_{i_r}.$$

Similarly, since $y_{i_{r-1}} > y_{i_r}$,

$$c_{i_r} = \frac{y_{i_r}}{x_{i_r} + y_{i_r}} < \frac{y_{i_{r-1}}}{x_{i_r} + y_{i_{r-1}}} = \alpha_{r-1}.$$

Thus

$$\alpha_r < c_{i_r} < \alpha_{r-1},$$

with the evident boundary modifications. Hence the integral over the active interval of i_r splits at the kink of τ_{i_r} .

Integral over one active interval

Fix r , and abbreviate

$$x = x_{i_r}, \quad y = y_{i_r}, \quad a = \alpha_r, \quad b = \alpha_{r-1}, \quad c = \frac{y}{x+y}.$$

Since $a \leq c \leq b$, the contribution of p_{i_r} to the lower-envelope integral is

$$\int_a^b \tau_{i_r}(w) dw = \int_a^c (1-w)y dw + \int_c^b wx dw.$$

Evaluating the two elementary integrals gives

$$\int_a^b \tau_{i_r}(w) dw = y \left(c - \frac{c^2}{2} - a + \frac{a^2}{2} \right) + x \left(\frac{b^2}{2} - \frac{c^2}{2} \right).$$

Collecting the terms involving c , we obtain

$$yc - \frac{(x+y)c^2}{2}.$$

Since $c = y/(x+y)$, this becomes

$$\frac{y^2}{x+y} - \frac{y^2}{2(x+y)} = \frac{y^2}{2(x+y)}.$$

Therefore

$$\int_a^b \tau_{i_r}(w) dw = \frac{xb^2}{2} - ya + \frac{ya^2}{2} + \frac{y^2}{2(x+y)}.$$

This expression is more useful when written in a symmetric boundary form. Observe that

$$-ya + \frac{ya^2}{2} + \frac{y}{2} = \frac{y(1-a)^2}{2},$$

and

$$\frac{y^2}{2(x+y)} - \frac{y}{2} = -\frac{xy}{2(x+y)}.$$

Hence

$$\int_a^b \tau_{i_r}(w) dw = \frac{xb^2}{2} + \frac{y(1-a)^2}{2} - \frac{xy}{2(x+y)}.$$

Returning to the original notation, this gives

$$\int_{\alpha_r}^{\alpha_{r-1}} \tau_{i_r}(w) dw = \frac{x_{i_r} \alpha_{r-1}^2}{2} + \frac{y_{i_r} (1 - \alpha_r)^2}{2} - A_{i_r i_r}.$$

Summation and telescoping

We now sum the active-interval contributions over all selected points:

$$R_2(i_1, \dots, i_k) = \sum_{r=1}^k \left[\frac{x_{i_r} \alpha_{r-1}^2}{2} + \frac{y_{i_r} (1 - \alpha_r)^2}{2} - A_{i_r i_r} \right].$$

The diagonal corrections are already in the desired form:

$$-\sum_{r=1}^k A_{i_r i_r}.$$

It remains to simplify the boundary terms containing the α_r . The first boundary value is $\alpha_0 = 1$, and hence

$$\frac{x_{i_1} \alpha_0^2}{2} = \frac{x_{i_1}}{2}.$$

The last boundary value is $\alpha_k = 0$, and hence

$$\frac{y_{i_k}(1 - \alpha_k)^2}{2} = \frac{y_{i_k}}{2}.$$

Now consider an interior switch between i_r and i_{r+1} . The two contributions meeting at this switch are

$$\frac{y_{i_r}(1 - \alpha_r)^2}{2} \quad \text{and} \quad \frac{x_{i_{r+1}}\alpha_r^2}{2}.$$

Using

$$\alpha_r = \frac{y_{i_r}}{x_{i_{r+1}} + y_{i_r}}, \quad 1 - \alpha_r = \frac{x_{i_{r+1}}}{x_{i_{r+1}} + y_{i_r}},$$

we obtain

$$\frac{y_{i_r}(1 - \alpha_r)^2}{2} + \frac{x_{i_{r+1}}\alpha_r^2}{2} = \frac{y_{i_r}x_{i_{r+1}}^2 + x_{i_{r+1}}y_{i_r}^2}{2(x_{i_{r+1}} + y_{i_r})^2}.$$

Factoring the numerator gives

$$\frac{x_{i_{r+1}}y_{i_r}(x_{i_{r+1}} + y_{i_r})}{2(x_{i_{r+1}} + y_{i_r})^2} = \frac{x_{i_{r+1}}y_{i_r}}{2(x_{i_{r+1}} + y_{i_r})}.$$

By definition, this is precisely

$$A_{i_r i_{r+1}}.$$

Collecting the two outer boundary terms, the interior switch terms, and the diagonal corrections, we arrive at

$$R_2(i_1, \dots, i_k) = \frac{x_{i_1}}{2} + \sum_{r=1}^{k-1} A_{i_r i_{r+1}} + \frac{y_{i_k}}{2} - \sum_{r=1}^k A_{i_r i_r}.$$

This proves the exact adjacent-neighbor decomposition used in the main text. The continuous integral over the lower envelope of the Tchebycheff shadows $\tau_i(w)$ depends, for a sorted selected chain, only on the two boundary points, the selected points through their diagonal corrections, and the adjacent selected pairs. This locality is the structural reason why the Bellman recurrence is exact.

B A gentle introduction to matrix search

Matrix search is a family of algorithms for finding all row minima or all column minima of a matrix faster than by inspecting every entry. The simplest way to view the technique is not as a classical linear algebra method but as an order-exploitation method: if the matrix has enough structure, many entries can be certified irrelevant without being evaluated. The classical reference is the matrix-search algorithm of Aggarwal, Klawe, Moran, Shor, and Wilber, usually referred to by their initials as SMAWK [9]. A broad survey of Monge properties in optimization is given by Burkard, Klinz, and Rudolf [10].

B.1 From Monge matrices to monotone minima

A matrix C is Monge if, for every $i < i'$ and $j < j'$,

$$C_{ij} + C_{i'j'} \leq C_{ij'} + C_{i'j}. \quad (12)$$

This inequality says that the north-west plus south-east diagonal of every feasible 2×2 submatrix is no larger than the other diagonal. For minimization, this has an important consequence. Suppose row i' is no worse than row i in column j , i.e.

$$C_{i'j} \leq C_{ij}.$$

Then (12) implies

$$C_{i'j'} - C_{ij'} \leq C_{i'j} - C_{ij} \leq 0,$$

so row i' is still no worse than row i in every later column j' . Thus, as one scans columns from left to right, the row index of the uppermost or leftmost column minimum cannot move upwards. This is the column-minimum form of total monotonicity. Rectangular SMAWK exploits this property recursively; the present paper uses a more specialized envelope form because the dynamic-programming matrix is only partially defined.

The distinction between Monge and totally monotone is useful. Monge is a strong 2×2 inequality. Total monotonicity is the weaker order condition needed by matrix search: minima remain ordered not only in the whole matrix but also in submatrices. Every Monge matrix is totally monotone, but not every totally monotone matrix is Monge.

B.2 A small rectangular example

Consider the matrix $C_{ij} = (i - j)^2$, with rows $i = 1, \dots, 4$ and columns $j = 1, \dots, 6$:

	c_1	c_2	c_3	c_4	c_5	c_6
r_1	0	1	4	9	16	25
r_2	1	0	1	4	9	16
r_3	4	1	0	1	4	9
r_4	9	4	1	0	1	4

The column-minimizing row indices are

$$1, 2, 3, 4, 4, 4.$$

They move only downward as the column index increases. For instance, row 3 beats row 2 at column 4, since $1 < 4$. From that column onward, row 2 can never again become better than row 3. This is exactly the kind of dominance that matrix search uses to discard candidates. The algorithm does not need to evaluate all entries in order to know where minima can still occur; it only has to preserve a small ordered set of plausible rows.

The example also illustrates the 2×2 Monge inequality. Using rows 2, 4 and columns 3, 5, one obtains

$$C_{2,3} + C_{4,5} = 1 + 1 = 2 \leq C_{2,5} + C_{4,3} = 9 + 1 = 10.$$

This inequality is the algebraic reason that a later row, once it has become better than an earlier row, remains better in later columns.

B.3 Why the dynamic-programming matrix is a staircase

The matrices in the Bellman recurrence are not full rectangles. In layer r , column j asks for a predecessor $i < j$, and enough points must have been selected before i . Hence the feasible entries are

$$\Omega_r = \{(i, j) : r - 1 \leq i < j \leq n\}.$$

As j increases, one new predecessor row becomes feasible. The feasible region is therefore a staircase. A tiny schematic example is

	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$i = 1$	*	*	*	*	*
$i = 2$		*	*	*	*
$i = 3$			*	*	*
$i = 4$				*	*

where stars denote feasible entries. Padding the blank entries by $+\infty$ is not the cleanest formalization, because artificial infinite ties may obscure the total-monotonicity argument. The right object is a partially defined staircase Monge matrix: every fully feasible 2×2 submatrix satisfies the Monge inequality, and feasible row sets are nested prefixes.

B.4 The envelope interpretation

For the integral R_2 recurrence, each predecessor row i in layer r defines the function

$$f_i(x) = D_{r-1}(i) + \frac{xy_i}{2(x + y_i)}.$$

The value required in column j is the lower envelope value

$$\min_{i < j} f_i(x_j).$$

Because the Pareto-front approximation is sorted with decreasing y_i , two such functions cross at most once. Thus the lower envelope is an ordered list of row intervals. The staircase scan performs three simple operations:

1. insert the new row $i = j - 1$ when it first becomes feasible;
2. delete previous rows whose future interval of optimality is destroyed by the new row;
3. query the current envelope at the next increasing abscissa x_j .

Each row is inserted once and deleted at most once. The query pointer over the envelope intervals also moves only forward. This is the same order principle as matrix search, specialized to the analytic transition functions of the integral R_2 dynamic program.

B.5 Relation to SMAWK matrix search

SMAWK is a general-purpose algorithm for rectangular totally monotone matrices under implicit access to entries. It alternates two ideas: eliminate columns that cannot contain a row minimum, and recursively solve a smaller problem before filling the gaps by local search. The algorithm in Section 4.4 does not need the full generality of rectangular SMAWK. Its matrices have a staircase feasible domain and rows enter in the same order as the columns are scanned. Moreover, the entries are generated by single-crossing functions with explicit crossing points. The lower-envelope sweep is therefore a direct, transparent instance of matrix search for this particular recurrence.

The practical message is that direct Bellman recurrence evaluates all feasible predecessor–endpoint pairs in a layer. Divide-and-conquer dynamic programming uses monotone predecessor indices and reduces the search range recursively. Matrix search goes one step further: it keeps only the rows that can still become optimal for some future endpoint. In the bi-objective integral R_2 setting, this reduces one layer to linear time while preserving the same exact Bellman recurrence and the same leftmost tie-breaking convention.

References

- [1] M. P. Hansen and A. Jaskiewicz. Evaluating the quality of approximations to the non-dominated set. Technical Report IMM-REP-1998-7, Department of Mathematical Modelling, Technical University of Denmark, 1998. https://www.imm.dtu.dk/documents/ftp/tr98/tr07_98.abstract.html.
- [2] L. Schäpermeier and P. Kerschke. Reinvestigating the R_2 indicator: Achieving Pareto compliance by integration. In *Parallel Problem Solving from Nature – PPSN XVIII*, pages 202–216. Springer, 2024. https://doi.org/10.1007/978-3-031-70085-9_13.
- [3] L. Schäpermeier and P. Kerschke. R_2 v2: The Pareto-compliant R_2 indicator for better benchmarking in bi-objective optimization. *Evolutionary Computation*, 2025. <https://doi.org/10.1162/EVC0.a.366>.
- [4] A. Jaskiewicz and P. Zielniewicz. Exact calculation and properties of the R_2 multiobjective quality indicator. *IEEE Transactions on Evolutionary Computation*, 29(4):1227–1238, 2025.
- [5] D. Brockhoff, T. Wagner, and H. Trautmann. R_2 indicator-based multiobjective search. *Evolutionary Computation*, 23(3):369–395, 2015. https://doi.org/10.1162/EVC0_a_00135.
- [6] T. Kuhn, C. M. Fonseca, L. Paquete, S. Ruzika, M. M. Duarte, and J. R. Figueira. Hypervolume subset selection in two dimensions: Formulations and algorithms. *Evolutionary Computation*, 24(3):411–425, 2016.
- [7] K. Bringmann, T. Friedrich, and P. Klitzke. Two-dimensional subset selection for hypervolume and epsilon-indicator. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (GECCO ’14)*, pages 589–596. ACM, 2014. <https://doi.org/10.1145/2576768.2598276>.
- [8] K. Korogi and R. Tanabe. Dynamic programming for the indicator-based subset selection problem with IGD, IGD⁺, R_2 , and NR2 in bi-objective optimization. In *Proceedings of the 39th Annual Conference of the Japanese Society for Artificial Intelligence*, article 4P2-OS-17b-04, 2025. https://www.jstage.jst.go.jp/article/pjsai/JSAI2025/0/JSAI2025_4P2OS17b04/_article/-char/en.
- [9] A. Aggarwal, M. M. Klawe, S. Moran, P. Shor, and R. Wilber. Geometric applications of a matrix-searching algorithm. *Algorithmica*, 2:195–208, 1987.
- [10] R. E. Burkard, B. Klinz, and R. Rudolf. Perspectives of Monge properties in optimization. *Discrete Applied Mathematics*, 70(2):95–161, 1996.
- [11] M. T. M. Emmerich. Preference-shaped expected hypervolume and R_2 improvement: Exact computation and monotonicity. arXiv preprint arXiv:2605.28746, 2026. <https://arxiv.org/abs/2605.28746>.
- [12] M. T. M. Emmerich. Polynomial-time Riesz-energy subset selection for ordered point sets on lines and ℓ_1 -staircases. arXiv preprint arXiv:2606.16946, 2026. <https://arxiv.org/abs/2606.16946>.